**Written examination date:** May 23th, 2025
**Course title:** Introduction to reinforcement learning and control
**Course number:** 02465
**Aids allowed:** All aids allowed
**Exam duration:** 4 hours

**Weighting:** The exam is divided into 3 parts:

- Multiple-Choice questions
- Conceptual questions
- Programming questions

The overall scores of each part contribute roughly equally towards the overall result. Each question in each part contribute equally towards the score of that part.

**Part I:** Questions 1-9 are multiple choice. The score of a correct answer is 3 points. The score of an incorrect answer is $-1$ points. The score of option $E$ or blank is 0 points.

**Part II and part III:** Each completed sub-task contribute towards your score.

**Preparing your handin:** The three parts are prepared as follows:

**Part I:** Edit the file `irlc/exam/exam2025spring/multiple_choice_answers.py`. Don't include calculations. Only answer with `'A'`, `'B'`, `'C'`, `'D'`, `'E'`.

**Part II:** Create a PDF file with your answers and justifications.

**Part III:** Program your answer in the `.py`-files indicated in the question and run `irlc/exam/exam2025spring/exam2025spring_tests_grade.py` to generate your `.token`-file.

**Handing in:** To hand in, you should upload the files:

- The `irlc/exam/exam2025spring/multiple_choice_answers.py`-file with your answer to part I
- The `.pdf`-file with your answers to part II
- The `.token`-file with your answers to part III
- The `irlc/exam/exam2025spring/question_control_pendulum.py` source file containing your solution
- The `irlc/exam/exam2025spring/question_inventory_cake.py` source file containing your solution
- The `irlc/exam/exam2025spring/question_simple_q.py` source file containing your solution

**Note on part II:** The main quantities asked for are highlighted as $f(x)$. Answer unambigiously, concisely, and if applicable with algebraic simplifications. Your final result must be clearly indicated at the end of your answer: $f(x) = 3\sin(x)$. To get credit, you must state the relevant theory and equations, and all relevant calculations must be included. Credit is not given for answers with missing or erroneous justifications.

**Note on part III:** To get started, move the folder `irlc/exam/exam2025spring` from the `.zip` file to the corresponding location in your existing exercise directory. The `.py` source files must be **reproducible** and **readable** so that someone else can run and fully understand your solution. You can freely use the `irlc`-toolbox (including solutions) and the packages we have used in the course, but you **must** include additional code you write during the exam or have prepared beforehand in the source files listed above. The source files must not be renamed. The `.token` file contains your results and must be up to date with your source files, i.e., generate it just prior to handin. In the case they differ, the `.token` file takes precedence. Credit is given for correct implementations defined by the problem description. The points in the `.token` file name is computed from the public tests, and might not correspond to overall correctness.

# Part I: Multiple-Choice

**Question 1:** .....

Consider a dynamical programming (DP) problem with dynamics $x_{k+1} = f_k(x_k, u_k, w_k)$ and cost functions $g_k(x_k, u_k, w_k)$ and $g_N(x_N)$. Recall that the DP algorithm initialize $J_N(x_N) = g_N(x_N)$ and iterate:

$$J_k(x_k) = \min_{u_k} \mathbb{E}_{w_k}[g_k(x_k, u_k, w_k) + J_{k+1}(f_k(x_k, u_k, w_k))], \quad k = N-1, N-2, \ldots, 1, 0 \tag{1}$$

We now consider various modifications to the problem/DP algorithm and whether these modifications affects the optimal policy as computed using dynamical programming. Which one of the following statements are true?

A. If we add a constant $c > 0$ to the terminal cost, $g_N(x_N) \leftarrow g_N(x_N) + c$ (for all $x_N \in \mathcal{S}_N$), this will not affect the optimal policy.

B. If we add a constant $c > 0$ to the dynamics, $f_k(x_k, u_k, w_k) \leftarrow f_k(x_k, u_k, w_k) + c$, this will not affect the optimal policy.

C. If the DP algorithm is re-formulated by exchanging the minimum and the expectation it will still determine the optimal policy. Therefore, the following is also a valid DP update rule:

$$J_k(x_k) = \mathbb{E}_{w_k}[\min_{u_k} (g_k(x_k, u_k, w_k) + J_{k+1}(f_k(x_k, u_k, w_k)))]$$

D. If we scale the terminal cost with a constant $c > 1$, $g_N(x_N) \leftarrow c g_N(x_N)$ (for all $x_N \in \mathcal{S}_N$), this will not affect the optimal policy.

E. Don't know.

**Solution: The correct answer is A.**

A. Intuitively, adding a constant to the terminal cost $g_N$ does not make any of the terminal states more desirable, thus this will not affect the optimal policy. This can also be seen from eq. (1) where the constant can be taken outside of the minimum.

B. This only makes sense when the states $x_k$ are numbers, and in this case it would clearly just change the problem.

C. This is simply not true. For starters, the expectation is with respect to the distribution $p(w_k|x_k, u_k)$ and thus the expectation could not be computed in this case.

D. Scaling the terminal cost does affect the optimal policy since it will affect how important the terminal state is relative to all other states. Mathematically, when $k = N-1$, the update rule eq. (1) changes to:

$$J_k(x_k) = \min_{u_k} \mathbb{E}_{w_k}[g_k(x_k, u_k, w_k) + c g_N(f_k(x_k, u_k, w_k))].$$

This can clearly affect what value of $u_k$ realize the minimum and thus the policy.

**Question 2:** .....

Maria is operating a nuclear reactor. Every second $k$, Maria observes the temperature of the reactor $x_k$, which she then use to compute a change (in millimeters) of the control rod position $u_k$.

- If $u_k < 0$ over a longer period the control rods are pushed in and the reactor will cool down and stop completely.
- If $u_k > 0$ over a longer period the control rods are pulled out and the reactor will heat up and eventually explode.

Question 2 continues on the next page...

Maria uses the PID controller from the course ([Her25, algorithm 19]) to control the reactor and has carefully tuned the $K_p > 0$, $K_i > 0$ and $K_d > 0$ parameters to keep the temperature at $x^* = 500$ degrees. The PID controller implements the control rule (using error $e = x^* - x_k$ and $I$ for the integral term):

$$u_k = K_p e + K_d \frac{e - e^{\text{prev}}}{\Delta} + K_i I$$

For example, if the temperature is too low ($x_k = 495$), the $K_p$ term alone will compute a control of $u_k = 5K_p$ (i.e., raise the control rods a bit).

Due to a mechanical defect, the control rods got stuck in a slightly too low position, which meant that the reactor was operating at a temperature of $x_k = 495$ the entire night. What happens the next morning when Maria fixes the mechanical defect so that the control rods can again begin to move according to the control rule?

A. Over night, the derivative term has taken a very large, negative value which will slam down the control rods and cause the reactor to shut down.

B. The integral term will be very large which will yank out the control rods.

C. The proportional term will take a very large, negative value, which will slam down the control rods and the reactor will shut down.

D. The control rule will effectively compute 0 control output for the first few steps, thus it will still operate at a too low temperature.

E. Don't know.

**Solution: The correct answer is B.**

As in the example the error is $e = x^* - x_k = 5$ degrees. This means that the proportional term is $5K_p$, the derivative term is a change in the error and thus 0, however, the integral term becomes the sum of all all errors seen during all $10 \times 60 \times 60 = 36000$ seconds of the night, and thus will be of the order $36000 \times 5 \times K_i$. Moreover, it will only change downwards slowly over time as new (negative) errors are added to it. Thus, the control signal becomes very large, removing the control rods and bad things will probably follow.

**Question 3:** .....

Bob wants to apply optimal control by direct collocation to a problem where the states are $n$-dimensional, $\boldsymbol{x}(t) \in \mathbb{R}^n$ and the actions are $d$-dimensional, $\boldsymbol{u}(t) \in \mathbb{R}^d$.

Bob wishes to determine the optimal policy for the problem using direct collocation with a grid-size of $N$, however, Bob remembers that the direct method works by creating a large optimization problem in the format $\min_{\boldsymbol{z}} E(\boldsymbol{z})$ where $\boldsymbol{z} \in \mathbb{R}^m$. Bob is worried that the problem is too large for his computer and thus he wishes to determine how the problem size scales. Which one of the following statements are true about the size $m$ of the optimization problem?

A. The size is approximately $n + d + N$.

B. The size is approximately $ndN$

C. The size is approximately $n^2N + d^2N$

D. The size is approximately $(n + d)N$

E. Don't know.

**Solution: The correct answer is D.**

Collocation discretize the problem by introducing the new variables $\boldsymbol{x}_k$ and $\boldsymbol{u}_k$ for $k = 0, \ldots, N-1$. Thus the scale is about $nN + dN = (n + d)N$ (in fact, it is exactly $(n + d)N$ when we account for the time variables).

**Question 4:** .....

Alice consider the following one-dimensional control problem where the goal is to control a state $x(t)$ using a control signal $u(t)$. In the standard notation of the course the dynamics is:

$$\dot{x}(t) = \sin(x(t)) + u(t) = f(x(t), u(t)).$$

Alice is particularly interested in controlling the problem when $x(t) \approx 0$ and $u(t) \approx 0$. Therefore, Alice decides to first discretize the problem using a step-size of $\Delta = 1$, and then she linearize the problem around the state/control values $\bar{x} = 0$ and $\bar{u} = 0$. Which one of the following options represents the linearized dynamics?

A. $x_{k+1} = x_k + u_k$

B. $x_{k+1} = 2x_k + u_k$

C. $x_{k+1} = u_k$

D. $x_{k+1} = x_k + u_k + 1$

E. Don't know.

**Solution: The correct answer is B.**

We first discretize the problem using $x_{k+1} = x_k + \Delta f(x_k, u_k)$. This gives:

$$x_{k+1} = x_k + \sin(x_k) + u_k = f_k(x_k, u_k)$$

Plugging this into the general formula for linearization gives us:

$$\begin{aligned} x_{k+1} &= f_k(\bar{x}, \bar{u}) + \frac{\partial f_k}{\partial x_k}(\bar{x}, \bar{u})(x_k - \bar{x}) + \frac{\partial f_k}{\partial u_k}(\bar{x}, \bar{u})(u_k - \bar{u}) \\ &= 0 + (1 + \cos(\bar{x}))(x_k - 0) + 1(u_k - 0) \\ &= 2x_k + u_k. \end{aligned}$$

Alternatively, remember that $\sin(x) \approx x$ for small values of $x$ and in this case the problem is already linear!

**Question 5:** .....

Consider the double integrator example described in [Her25, section 16.1.1]. In the example, $\boldsymbol{x}_k$ is two-dimensional and satisfy the discrete dynamics: $\boldsymbol{x}_{k+1} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \boldsymbol{x}_k + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u_k$. The cost-function is modified by adding a constant $d > 0$, but otherwise we use the same type of cost-matrices as in the original problem. The cost is therefore:

$$J^d(\boldsymbol{x}_0) = \sum_{k=0}^{N-1} c(\boldsymbol{x}_k, \boldsymbol{u}_k), \qquad \text{where} \qquad c(\boldsymbol{x}_k, \boldsymbol{u}_k) = \frac{1}{2}\boldsymbol{x}_k^\top \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \boldsymbol{x}_k + \frac{1}{2}u_k^2 \underbrace{+d}_{\text{new term!}}$$

Let $\pi^d$ be the optimal policy for this problem computed using discrete LQR.

Which one of the following statements is correct when we apply LQR on a long horizon $N \approx 1000$

A. When the policy $\pi^d$ is executed, using cost function $J^d$, then after a handful of steps the agent will obtain a cost of about $kd$ per step.

B. When the policy $\pi^d$ is executed, using cost function $J^d$, then after a handful of steps the agent will obtain a cost of about $d$ per step.

C. $d$ affects the optimal policy. Specifically, the policies $\pi^{d=10}$ will tend to compute **smaller** actions than $\pi^0$.

D. $d$ affects the optimal policy. Specifically, the policies $\pi^{d=10}$ will tend to compute **larger** actions than $\pi^0$.

E. Don't know.

**Solution: The correct answer is B.**

Adding a constant to the cost-function does not affect the optimal policy (which is the minimum of the cost function).

As discussed in the lecture and section 16.1.1, after a few steps $k > 10$ the double integrator will converge to the solution where $\boldsymbol{x}_k \approx 0$ and subsequently apply an action of $u_k = 0$. After this point, the only contribution to the cost function per step is $d$.

**Question 6:** .....

Consider the simple bandit algorithm applied to the 10-armed testbed described in [SB18, Chapter 2] for 1000 steps using $\varepsilon = 0.1$. Recall that in the 10-armed testbed, the average rewards for arm $i = 1, \ldots, k$ is i.i.d. normally distributed with mean 0:

$$q^*(i) \sim \mathcal{N}(\mu = 0, \sigma^2 = 1).$$

Which one of the following statements are true?

A. Assume that we train the simple bandit method for 1000 steps and let $R_{1000}$ be the last reward we receive. It is more likely that $R_{1000}$ is positive than negative.

B. Suppose that all arms have the same average reward, i.e. $q^*(1) = q^*(2) = \cdots = q^*(k)$. Then after $T = 1000$ steps they will all have been tried the same number of times, $N(1) = N(2) = \cdots = N(k)$.

C. During the first $T = 1000$ steps, it is *not* possible that the estimated action-value are smaller than their true values, i.e. it *cannot* be the case that $Q_t(a) < q^*(a)$.

D. During the first $T = 1000$ steps, it is *not* possible that the estimated action-value are greater than their true values, i.e. it *cannot* be the case that $Q_t(a) > q^*(a)$.

E. Don't know.

**Solution: The correct answer is A.**

A. Is is very likely that at least one arm has a positive reward $q^*(k) > 0$. If this is the case then the optimal arm has an average reward greater than 0, and if we assume the simple bandit algorithm works it is thus likely that we select the optimal arm most of the time, and most of the time get a positive reward. Another way to see this is from e.g. [SB18, Figure 2.2] where we see that $\mathbb{E}[R_{1000}] \approx 0.8$.

B. In this case the algorithm is basically random: The algorithm selects random arms with probability $\varepsilon$, and otherwise it selects the arm which (by coincidence!) happen to have the higest estimated $Q_t(a)$-value. It is exceedingly unlikely that all $N(k)$-values are equal.

C. In the long run we expect that $Q_t(a) \to q^*(a)$ for the optimal arm, but it is not possible to say how. There is equal chance of over/under estimation.

D. In the long run we expect that $Q_t(a) \to q^*(a)$ for the optimal arm, but it is not possible to say how. There is equal chance of over/under estimation.

**Question 7:** .....

Consider the Mountain Car example described in [SB18, Example 10.1]. Recall that the state $s = \begin{bmatrix} s_1 \\ s_2 \end{bmatrix}$ is two-dimensional. As described in the example, the actions are $a = -1$, $a = 0$, and $a = 1$ corresponding to reverse throttle, no throttle and forward throttle respectively.

Let's assume that we wish to solve the problem using semi-gradient Sarsa using linear function approximators. We select the following feature-vector:

$$\boldsymbol{x}(s, a) = \begin{bmatrix} a s_1 \\ a^2 s_2^2 \\ a s_1 s_2 \end{bmatrix} \in \mathbb{R}^3$$

and the usual approximation of the $q$-function as $\hat{q}(s, a; \boldsymbol{w}) = \boldsymbol{x}(s, a)^\top \boldsymbol{w}$. For plotting purposes, we define the (approximate) optimal value function from the $\hat{q}$-functions as:

$$\hat{v}(s; \boldsymbol{w}) = \max_a \hat{q}(s, a; \boldsymbol{w}).$$

Question 7 continues on the next page...

Suppose that Sarsa has learned the feature vector $\boldsymbol{w} = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}$, what is $\hat{v}(s; \boldsymbol{w})$ in $s = \begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \end{bmatrix}$?

A. $\hat{v}(s, \boldsymbol{w}) = -\frac{3}{4}$

B. $\hat{v}(s, \boldsymbol{w}) = 2$

C. $\hat{v}(s, \boldsymbol{w}) = \frac{1}{2}$

D. $\hat{v}(s, \boldsymbol{w}) = 1$

E. Don't know.

**Solution: The correct answer is D.**

In the given state we have that

$$\boldsymbol{x}(s, a)^\top w = \begin{bmatrix} \frac{1}{2}a \\ \frac{1}{4}a^2 \\ -\frac{1}{4}a \end{bmatrix}^\top \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} = \frac{1}{2}a + \frac{1}{2}a^2 - a = \frac{1}{2}a^2 - \frac{1}{2}a$$

So therefore by trying all the actions:

$$\hat{v}(s; \boldsymbol{w}) = \max_{a=-1,0,1} \hat{q}(s, a; \boldsymbol{w}) = \max \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = 1.$$

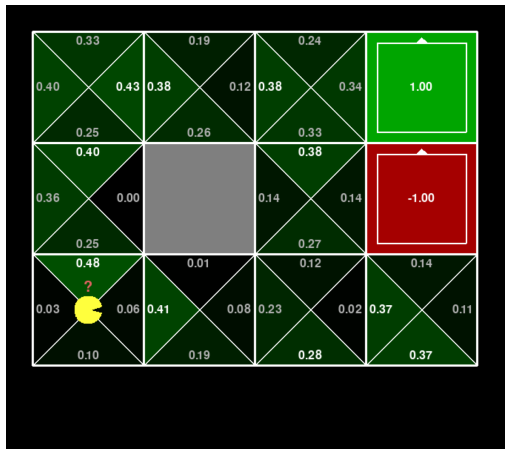


Figure 1: The familiar gridworld environment from with randomly initialized $Q$-values. The living reward is $-0.05$, and the two squares in the upper-right corner (with $Q$-value of $+1$ and $-1$) has a single action that lead to the terminal state where the agent obtains a reward of $+1$ and $-1$ respectively.

**Question 8:** .....

We consider the familiar gridworld environment discussed in [Her25, section 4.2.4] shown in fig. 1.

Suppose that **Sarsa-learning** (using discount factor $\gamma = 0.6$ and learning rate $\alpha = 0.9$) is applied to the Gridworld MDP shown in fig. 1. The living reward is $R_{t+1} = -0.05$, and the dynamics are deterministic. The current state $s_t$ is indicated by Pacman in the figure.

The $Q$-values have been initialized randomly, however, this does not alter how Sarsa-learning functions. Assuming that from the given position, the agent will take the actions `North` and then `North`.

After these two actions, what will be the new value of the $Q$-value indicated by the red question mark?

A. 0.219

B. 0.19

C. 0.699

D. 0.237

E. Don't know.

**Solution: The correct answer is A.**

The solution can be found by applying the Q-learning update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R_t + \gamma Q(s',a') - Q(s,a))$$

By inspecting the figure, we see that (originally) $Q(s,a) = 0.48$, $Q(s',b') = 0.4$ and therefore the updated value is $Q(s,a) = 0.219$.

**Question 9:** .....

Consider again the gridworld problem described in fig. 1, but assume that $\gamma = 1$ (i.e., there is no discount factor). The starting state is indicated by Pacman.

Suppose that we apply Sarsa($\lambda$) to the problem using $0 < \lambda < 1$. However, we apply Sarsa($\lambda$) in such a way that in each episode, we ensure that the agent always generate actions corresponding to taking the shortest path to goal square in the upper-right corner: The agent always walk north to the upper-left corner, and then walk due east to the goal in the upper-right corner. This means that all episodes will be the same.

Question 9 continues on the next page...

Under these circumstances Sarsa($\lambda$) will still update the $Q$-values on the agents path. Assuming we have selected a small learning rate $\alpha$ which guarantee convergence (or equivalently that $\alpha \to 0$), what will the $Q$-value indicated by the red question mark, $Q(s, a)$, become after many episodes?

A. $Q(s, a) \approx 0.75$

B. Since we don't know what $\lambda$ is the problem cannot be answered.

C. $Q(s, a) \approx 1$

D. $Q(s, a) \approx 0.95$

E. Don't know.

**Solution: The correct answer is A.**

Let's call the north-east policy $\pi$. Since Sarsa-$\lambda$ is assumed to converge, Sarsa-$\lambda$ will perform policy evaluation, i.e., it will learn the $q$-function associated with the north-east policy $\pi$.

It is helpful that $\gamma = 1$ since this means that the $q$-function is simply the sum of reward:

$$Q(s, a) = \sum_{t=0}^{T-1} R_{t+1} = -0.05 - 0.05 - 0.05 - 0.05 - 0.05 + 1 = 0.75.$$

Note that $\lambda$ is irrelevant for the problem.

# Part II: Conceptual questions

**Question 10:** .....

Consider a control problem where a control signal $u(t) \in \mathbb{R}$ is applied to control a system with state $x(t) \in \mathbb{R}$.

Using the standard notation (see [Her25, eq. (10.12)]), the cost function is an (integral) of the instantaneous cost

$$c(t, x, u) = (x - 2)^2 + u^2 + 1 \tag{2}$$

plus a terminal cost term which is set to zero $c_F = 0$. This gives a cost function of a trajectory $x(t)$ and $u(t)$ of

$$J_u(x, t_0, t_F) = \int_{t_0}^{t_F} c(\tau, x(\tau), u(\tau)) d\tau$$

(a) Assume that all trajectories $x(t)$ and $u(t)$ are possible. What is the minimum value the cost function can take when evaluated from $t_0 = 0$ to $t_F = T$, i.e., $\min_{x,u} J_u(x, 0, T)$? *Hint: Your answer should be a function of $T$.*

**Solution:** We can select $x(t) = 2$ and $u(t) = 0$. In this case the integral is $\int_0^T 1 d\tau = T$.

(b) Now assume that the dynamics satisfy the following differential equation:

$$\dot{x}(t) = x(t) + u(t), \quad 0 \leq t \leq T. \tag{3}$$

and we apply a control signal of the form $u(t) = k_p x(t)$ starting in $x(0) = 2$. Determine the state $x(t)$ as a function of time when this control signal is applied. Your answer should be simplified to avoid e.g. integrals. *Hint: The solution will be of the form $x(t) = be^{at}$.*

**Solution:** Inserting and differentiating we get

$$ax(t) = x(t) + k_p x(t) = (1 + k_p)x(t)$$

Therefore we conclude that $a = (1 + k_p)$. Similarly inserting $x(0) = 2$ we get that $2 = x(0) = be^0$ and therefore $b = 2$. All in all

$$x(t) = 2e^{(1+k_p)t}$$

(c) Continuing the previous problem but assuming that the instantaneous cost function is instead:

$$c(t, x, u) = u^2$$

Determine the cost function $J_u(x, 0, T)$ as a function of $T$ when $k_p = -2$ and using the dynamics eq. (3). Your answer should be a simple expression and not involve e.g. an integral.

**Solution:** When $k_p = -2$ then $x(t) = 2e^{-t}$ and therefore $u(t) = k_p x(t) = -4e^{-t}$. Therefore the cost is:

$$J = \int_0^T u(t)^2 dt = 16 \int_0^T e^{-2t} dt = 16 \frac{-1}{2}(e^{-2T} - 1) = 8(1 - e^{-2T})$$

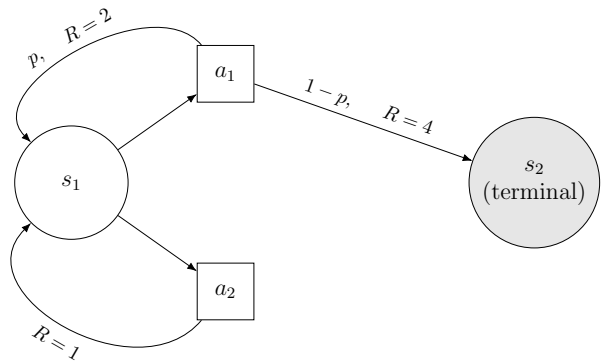


Figure 2: A Markov Decision Problem with non-terminal state $s_1$, two actions $a_1$ and $a_2$, and a terminal state $s_2$.

**Question 11:** .....

Consider a Markov Decision Process with two states $s_1$ and $s_2$ and a discount factor of $0 < \gamma \leq 1$ shown in fig. 2.

- The initial state is always $s_1$ and the state $s_2$ is terminal.
- In the state $s_1$, there are two actions $a_1$ and $a_2$.
- If we take action $a_1$:
  - With a probability $0 < p < 1$ we will transition to $s_1$ (i.e., stay in $s_1$) and obtain a reward of 2
  - With a probability of $1 - p$, we will transition to $s_2$, obtain a reward of 4, and terminate
- If we take action $a_2$ we will always transition to $s_1$ and always obtain a reward of 1.

Suppose that during an episode the player starts in $s_1$, and performs the following action sequence:

$$A_0 = a_2, \quad A_1 = a_1, \quad A_2 = a_2, \quad A_3 = a_1$$

After the last action, the player transitioned to the state $S_4 = s_2$ and the problem terminated. From the given dynamics, the rewards the player obtained must have been:

$$R_1 = 1, \quad R_2 = 2, \quad R_3 = 1, \quad R_4 = 4$$

(a) Using first-visit Monte-Carlo for value estimation, what is the estimated value of $v(s_1)$ based on this single episode? Provide your answer as a function of $\gamma$ and **also** include the numerical value when $\gamma = \frac{1}{2}$.

**Solution:**

$$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 = 1 + 2\gamma + \gamma^2 + 4\gamma^3.$$

Therefore when $\gamma = \frac{1}{2}$:

$$v_\pi(s_1) = 1 + 2\frac{1}{2} + \frac{1}{4} + 4\frac{1}{8} = 2 + \frac{3}{4} = 2.75.$$

(b) Assume that $\gamma = 1$. Using every-visit Monte-Carlo for action-value estimation, determine the numerical value of the action-value function $q(s_1, a_1)$ based on this single episode.

**Solution:** We take action $a_1$ twice. The first time the return is:

$$G_1 = R_2 + R_3 + R_4.$$

The second time, at time $t = 3$, it is:

$$G_3 = R_4.$$

Therefore

$$q_\pi(s_1, a_1) = \frac{G_1 + G_2}{2} = \frac{R_2 + R_3 + 2R_4}{2} = \frac{2 + 1 + 8}{2} = \frac{11}{2} = 5.5.$$

**Question 12:** .....

This problem again deals with the two-state MDP described in the previous problem (see fig. 2). Recall that $0 < \gamma \leq 1$ and $0 < p < 1$.

(a) Let $\pi$ be a policy which always selects action $a_1$, i.e. $\pi(a_1|s_1) = 1$. Determine the value function $v_\pi(s_1)$ as a function of $p$ and $\gamma$ and **also** include the numerical value when $p = \frac{1}{2}$ and $\gamma = 1$. *(Hint: Use a Bellman equation).*

**Solution:** The bellman equation for this state is (see [SB18, equation 3.14]):

$$v_\pi(s_1) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_1] = \sum_a \pi(a|s_1) \sum_{s',r} p(s', r|s_1, a)(r + \gamma v_\pi(s)) \tag{4}$$

Since we select action $a_1$ with certainty, we can either stay in $s_1$ or transition to $s_2$ where $v_\pi(s_2) = 0$ since it is a terminal state. Therefore:

$$v_\pi(s_1) = p(2 + \gamma v_\pi(s_1)) + (1 - p)(4 + 0) = 4 - 2p + p\gamma v_\pi(s_1) \tag{5}$$

Solving for $v_\pi(s_1)$ we get:

$$(1 - \gamma p)v_\pi(s_1) = 4 - 2p$$

Therefore $v_\pi(s_1) = \frac{4-2p}{1-\gamma p}$ and for $p = \frac{1}{2}$, $\gamma = 1$: $v_\pi(s_1) = 6$.

(b) Assume that $p = \frac{1}{2}$. Determine the numerical value of the value function $v_*(s_1)$ associated with the optimal policy **both** when (1) $\gamma = \frac{3}{4}$ **and** when (2) $\gamma = \frac{5}{6}$. *(Hint: You can use the previous answer. The optimal policy is deterministic which means that there are only two relevant policies).*

**Solution:** By definition, the optimal value function is:

$$v_*(s_1) = \max_\pi v_\pi(s_1)$$

There are two possible policies: One of which always selects $a_1$, and the other which always selects $a_2$. According to the previous problem the former had the value function:

$$v_\pi(s_1) = \frac{4 - 2p}{1 - \gamma p} = \frac{6}{2 - \gamma}$$

The other policy $\pi'$ we should consider is one that always selects action $a_2$. In this case the Bellman equation eq. (4) is simpler:

$$v_{\pi'}(s_1) = (1 + \gamma v_{\pi'}(s_1))$$

And therefore $v_{\pi'}(s_1) = \frac{1}{1-\gamma}$. Therefore $v_*(s_1) = \max\{\frac{1}{1-\gamma}, \frac{6}{2-\gamma}\}$. Inserting we get that:

$$\gamma = \frac{3}{4}: \quad v_*(s_1) = \max\{4, \frac{24}{5}\} = 4.8,$$
$$\gamma = \frac{5}{6}: \quad v_*(s_1) = \max\{6, \frac{36}{12 - 5}\} = 6.$$

Question 12 continues on the next page...

(In general, we see that $\pi'$ is preferred when $\gamma \geq \frac{4}{5}$).

# Part III: Programming questions

**Question 13:** .....

**To solve this question, you should edit the file `irlc/exam/exam2025spring/question_control_pendulum.py`.** In this problem, we will consider the Pendulum-swingup task described in [Her25, section 10.4.1]. Recall that the state of the pendulum is two-dimensional:

$$\boldsymbol{x} = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix}.$$

If we select particular values of the mass, gravitational constant and pendulum arm length, the state satisfy the following differential equation:

$$\ddot{\theta} = 1.25u + 9.82 \sin(\theta). \tag{6}$$

where $u$ is the control signal. Recall that to discretize the pendulum model, the first step is to re-write the dynamics eq. (6) into a first-order ordinary differential equation:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}(\boldsymbol{x}(t), u(t)) = \begin{bmatrix} \dot{\theta}(t) \\ 1.25u(t) + 9.82 \sin(\theta(t)) \end{bmatrix}. \tag{7}$$

Thus, the function $\boldsymbol{f}(\boldsymbol{x}, u)$ should return a two-dimensional vector which we denote $\begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$.

(a) Complete `def a_dynamics_f(theta, thetadot, u)`: Given a value of $\theta$ (as `theta`), $\dot{\theta}$ (as `thetadot`) and control signal $u$ (as `u`), the function should evaluate the dynamics $\boldsymbol{f}(\boldsymbol{x}, u)$ defined in eq. (7) and return the output vector $\begin{bmatrix} f_1 \\ f_2 \end{bmatrix}$ as a `list` with two elements: `[f1, f2]`, where `f1` and `f2` are both `float` numbers.

(b) Complete `def b_euler(theta0 : float, thetadot0 : float, delta : float, N : int)`: Assume that the control is $\boldsymbol{u}(t) = 0$ and that the dynamics eq. (7) is Euler discretized using a discretization time step $\Delta$ which gives the states

$$\boldsymbol{x}_0,\ \boldsymbol{x}_1, \ldots, \boldsymbol{x}_N.$$

Given an initial state $\boldsymbol{x}_0 = \begin{bmatrix} \theta_0 \\ \dot{\theta}_0 \end{bmatrix}$ (specified as `theta0` and `thetadot0`), a value of $\Delta$ (as `delta`), and $N$ (as `N`), the function should compute and return the first coordinate of the final state $\boldsymbol{x}_N$ as a `float`. This coordinate will correspond to the angle $\theta$ predicted using Euler discretization.

*Hint: This means that if $N = 0$, the function should return `theta0`, and if $N = 1$ it should perform exactly one Euler update and return the first coordinate of $\boldsymbol{x}_1 = \boldsymbol{x}_0 + \Delta \boldsymbol{f}(\boldsymbol{x}_0, 0)$, and so forth.*

**Solution:**

The solution can be found in this directory as a `.py` file.

**Question 14:** .....

**To solve this question, you should edit the file** `irlc/exam/exam2025spring/question_inventory_cake.py`**.** Bert the baker wants to plan how many cakes he should bake each day. Berts cake-store problem is a variant of the inventory control problem discussed in [Her25, section 5.1.2] where we plan on a horizon of $N$, and $x_k$ denote how many cakes are in stock in planning round $k$. Recall that:

- The customer demand is

$$P_k(w_k = 0|x_k, u_k) = \frac{1}{10}, \quad P_k(w_k = 1|x_k, u_k) = \frac{7}{10}, \quad \text{and} \quad P_k(w_k = 2|x_k, u_k) = \frac{1}{5}.$$

- The cost function is still $g_k(x_k, u_k, w_k) = u_k + (x_k + u_k - w_k)^2$.

(a) Complete `def a_expected_cost(x0 : int, u0 : int)`: Bert wishes to know how much profit he can expect in a single day on average. Given a starting state $x_0$ (as `x0`), and an action $u_0$ (as `u0`) this function should return a `float` corresponding to the expected cost obtained in the first planning round, i.e.:

$$\mathbb{E}_{w_0}[g_0(x_0, u_0, w_0)] = \sum_{w_0=0}^{2} P_0(w_0|x_0, u_0) g_0(x_0, u_0, w_0).$$

The function should compute this value for any positive integer values of $x_0 \in \{0, 1, \dots\}$ and $u_0 \in \{0, 1, \dots\}$.

(b) Complete `def b_best_action(N : int, cost_per_cake : float, k : int, x : int)`: The number of cakes Bert can sell in round $k$ given a demand of $w_k$ is:

$$\texttt{sold_cakes} = \min(w_k, x_k + u_k).$$

In the next two questions we consider a variant of the inventory-control problem where the cost-function is changed to[1]

$$g_k(x_k, u_k, w_k) = cu_k - \texttt{sold_cakes}, \quad g_N(x_N) = x_N^2$$

where $c$ is the cost per cake (`cost_per_cake`). This cost-function attempts to measure the profit and penalize a large inventory at the end of the planning period. Asides this change, the problem remains as in [Her25, section 5.1.2].

The function should compute and return the optimal action $\mu_k^*(x_k)$ as an `int` computed using $N$ rounds (given as `N`) of DP planning and given $k$ and $x_k$ (specified as `k` and `x` respectively).

(c) Complete `def c_lazy_baker(N : int, cost_per_cake : float, x0 : int)`: Continuing the previous problem, Bert decides to always bake 2 cakes on even days and not bake anything on odd days, i.e., Bert will now follow the policy:

$$u_k = \begin{cases} 2 & \text{if } k \text{ is even} \\ 0 & \text{otherwise} \end{cases}$$

This means the first action is $u_0 = 2$, the second is $u_1 = 0$, and so on. The function should compute the expected cost function $J_\pi(x_0)$ as a `float` in starting state $x_0$ (given as `x0`) when this policy is being followed over $N$ (given as `N`) planning rounds[2].

**Solution:**

The solution can be found in this directory as a `.py` file.

[1] Please notice the minus-sign!

[2] Hint: Your function should compute a non-random answer. A way to solve the problem is to limit the available action-spaces $\mathcal{A}_k$ so there is just one possible policy, and use DP to determine $J_0^*(x_0)$.

**Question 15:** .....

**To solve this question, you should edit the file** `irlc/exam/exam2025spring/question_simple_q.py`**.** This problem deals with the $Q$-learning method. We consider a problem where every state $s$ has exactly 2 actions, $\mathcal{A} = \{0, 1\}$, and the states $\mathcal{S}$ are integers $\{0, 1, 2, \dots\}$. There are no terminal states in the problem (i.e., it is not episodic).

**Datastructure:** The $Q$-values will be stored in a dictionary such that `Q[s,a]` store the $Q$-value $Q(s, a)$. For instance, the update:

$$Q(s, a) \leftarrow \frac{1}{2} Q(s, a) + 1$$

can be written as `Q[s,a] = 0.5 * Q[s,a] + 1`.

As usual, all $Q$-values that have not yet been updated/observed should be assumed to be initialized to zero[3]: $Q(s, a) = 0$

(a) Complete `def a_greedy_policy(q_values : dict, state : int)`: Given $Q$-values `q_values` as a dictionary in the previously described format and a state `state`, this function should compute and return the greedy action $a^* = \arg\max_{a \in \mathcal{A}} Q(s, a)$ as an `int`.

(b) Complete `def b_update_single_q(alpha, gamma, q_values: dict, state : int, action : int, reward : float, next_state : int)`: According to [SB18][Section 6.5], a single $Q$-learning update is of the form:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

The arguments of the function are $\alpha$, $\gamma$ (as `alpha`, `gamma`) and a dictionary of $Q$-values `q_values` in the format above. The next arguments are $S_t, A_t, R_{t+1}$ and $S_{t+1}$ (given as `state`, `action`, `reward` and `next_state`). The function should compute and return the updated $Q$-value $Q(S_t, A_t)$ as a `float`.

(c) Complete `def c_update_all_q(alpha, gamma, states_actions_rewards: list[tuple])`: In general, $Q$-learning is trained on a trajectory which consists of states, actions and rewards that are collected in the environment. These are stored in a `list` in the format:

$$(S_0, A_0, R_1),\ (S_1, A_1, R_2),\ (S_2, A_2, R_3),\ \dots,\ (S_{T-1}, A_{T-1}, R_T)$$

This mean that in the example input

```python
# question_simple_q.py
example_trajectory = [(0, 1, 0.5),   # s_0 = 0, a_0 = 1, r_1 = 0.5
                      (2, 0, -0.75), # s_1 = 2, a_1 = 0, r_2 = -0.75
                      (0, 1, 0.5),   # s_2 = 0, a_2 = 1, r_3 = 0.4
                      (1, 0, 0.5)]   # s_3 = 1, a_3 = 0, r_4 = -0.75
```

The first state is $S_0 = 0$, the first action taken is $A_0 = 1$, and the reward the agent immediately obtain is then $R_1 = 0.5$, and the length is $T = 4$.

The function is given a value of $\alpha$, $\gamma$ and a list of $T$ states/actions/rewards in the above format (as `alpha`, `gamma` and `states_actions_rewards` respectively). It should then update the $Q$-values based on the $T - 1$ $Q$-updates that can be computed given the input trajectory[4]. The function should return the updated $Q$-values as a dictionary using the same `Q[s,a]` format used in the previous problems.

**Solution:**

The solution can be found in this directory as a `.py` file.

[3] Hint: You can use `in` to check if an element is in a dictionary. For instance: `if (s, a) in Q: print('Q[s,a] exists!')`.

[4] It is only possible to apply $T - 1$ updates since we need to know $S_{t+1}$ to update $S_t$.

Question 15 continues on the next page...

## References

[Her25] Tue Herlau. Sequential decision making. (Freely available online), 2025.

[SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018. (Freely available online).

*This line concludes the exam. Document build: 2026/04/16, 22:36:00.*